| Course Title | AI Assisted Coding |
|---|---|

Name: A. Dinesh Roll Number: 2303A52329 Batch: 45

**Assignment Number: 13.5**(Present assignment number)/**24**(Total number of assignments)

Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from the code.

• Instructions:

o Identify global variables used across functions.

o Refactor the code to pass values using function parameters.

o Improve modularity and testability.

• Sample Legacy Code:

```
rate = 0.1

def calculate_interest(amount):

   return amount * rate

print(calculate_interest(1000))
```

• Expected Output:

o Refactored version passing rate as a parameter or using a configuration structure.

Prompt: Eliminate unnecessary global variables from the code.

Identify global variables used across functions.

Refactor the code to pass values using function parameters.

Improve modularity and testability.

Code:

```
def calculate_interest(amount, rate):
    """Calculate interest using an explicit rate parameter."""
    return amount * rate

def format_currency(amount):
    return f"${amount:,.2f}"

def main(amount=1000, rate=0.1):
    interest = calculate_interest(amount, rate)
    print(format_currency(interest))

if __name__ == "__main__":
    main()
rate = 0.1
```

Output:

```
tensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\la
.1.py'
$100.00
PS D:\3-2\AI Assitant Coding\Code Files> & "d:\3-2\AI Assitant Coding\
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>
```

Task Description #2 :  (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

• Focus Areas:

o Readability

o Logical simplification

o Maintainability

Legacy Code:

```
score = 78
if score >= 90:
    print("Excellent")
else:
    if score >= 75:
        print("Very Good")
    else:
        if score >= 60:
            print("Good")
        else:
            print("Needs Improvement")
```

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based approach.

Prompt: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

• Focus Areas:

o Readability

o Logical simplification

o Maintainability

Code:

```python
score = 78

def evaluate_score(score):
    thresholds = (
        (90, "Excellent"),
        (75, "Very Good"),
        (60, "Good"),
        (0, "Needs Improvement"),
    )
    for threshold, label in thresholds:
        if score >= threshold:
            return label

print(evaluate_score(score))
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\
cripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-20
' 'D:\3-2\AI Assitant Coding\Code Files\Task-13.1.py'
Very Good
PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 3 (Refactoring Repeated File Handling Code)

• Task: Use AI to refactor repeated file open/read/close logic.

• Focus Areas:

o DRY principle

o Context managers

o Function reuse

Legacy Code:

f = open("data1.txt")

print(f.read())

f.close()

f = open("data2.txt")

print(f.read())

f.close()

Expected Outcome:

o Reusable function using with open() and parameters.

Prompt: Refactor repeated file open/read/close logic.

• Focus Areas:

o DRY principle

o Context managers

o Function reuse

Code:

```
def read_and_print(path):
    with open(path, "r", encoding="utf-8") as f:
        print(f.read())
for filename in ("Task-10-3.py", "Task-1.py"):
    read_and_print(filename)
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\C
cripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-202
' 'D:\3-2\AI Assitant Coding\Code Files\Task-13.1.py'
# 25/02/26

"""Problem Statement 1: AI-Assisted Bug Detection
Scenario: A junior developer wrote the following Python function to calculate

def factorial(n):
    result = 1
    for i in range(1, n):
        result = result * i
    return result
Instructions:
1.      Run the code and test it with factorial(5).
2.      Use an AI assistant to:
o       Identify the logical bug in the code.
o       Explain why the bug occurs (e.g., off-by-one error).
o       Provide a corrected version.
```

Task 4 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

  if u == name:

    found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification.

Prompt: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Use of sets or dictionaries with complexity justification.

Code:

```python
def is_allowed_user(name: str, users_set: set) -> bool:
    # Set membership is average O(1) vs O(n) for lists
    return name in users_set

if __name__ == "__main__":
    USERS = {"admin", "guest", "editor", "viewer"}  # use a set for fast lookups
    name = input("Enter username: ").strip()
    print("Access Granted" if is_allowed_user(name, USERS) else "Access Denied")
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\
cripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-20
' 'D:\3-2\AI Assitant Coding\Code Files\Task-13.1.py'
Enter username: admin
Access Granted
PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

Prompt: Refactor procedural code into a class-based design.

• Focus Areas:

o Object-Oriented principles

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

Code:

```python
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
class EmployeeSalaryCalculator:
    def __init__(self, salary: float, tax_rate: float = 0.2):
        self.salary = float(salary)
        self.tax_rate = float(tax_rate)
    def calculate_tax(self) -> float:
        return self.salary * self.tax_rate
    def net_salary(self) -> float:
        return self.salary - self.calculate_tax()
    def formatted_net(self) -> str:
        return f"${self.net_salary():,.2f}"
if __name__ == "__main__":
    calc = EmployeeSalaryCalculator(50000)
    print(calc.formatted_net())
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\
cripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-20
' 'D:\3-2\AI Assitant Coding\Code Files\Task-13.1.py'
40000.0
$40,000.00
PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

```
total = 0
for i in range(1, 1000000):
    if i % 2 == 0:
        total += i
print(total)
```

Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Prompt: Refactor a performance-heavy loop handling large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison

Code:

```
import time
N = 1_000_000
def original_loop(n):
    total = 0
    for i in range(1, n):
        if i % 2 == 0:
            total += i
    return total
def comprehension(n):
    return sum(range(2, n, 2))
def formula(n):
    m = (n - 1) // 2  # number of even terms in range(1, n)
    return m * (m + 1)  # sum of first m evens = m(m+1)
if __name__ == "__main__":
    results = {}
    for fn in (original_loop, comprehension, formula):
        t0 = time.perf_counter()
        results[fn.__name__] = (fn(N), time.perf_counter() - t0)
    # print values and timings
    for name, (val, dt) in results.items():
        print(f"{name:15s}: {val} (t={dt:.6f}s)")
    # sanity check
    vals = {v for v, _ in results.values()}
    assert len(vals) == 1, "Results differ!"
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\
cripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-20
' 'D:\3-2\AI Assitant Coding\Code Files\Task-13.1.py'
original_loop  : 249999500000 (t=0.084664s)
comprehension  : 249999500000 (t=0.008533s)
formula        : 249999500000 (t=0.000004s)
PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Legacy Code:

```
data = []

def add_item(x):

    data.append(x)

add_item(10)

add_item(20)

print(data)
```

Expected Outcome:

o Refactored function returning values instead of mutating globals.

Prompt: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Expected Outcome:

o Refactored function returning values instead of mutating globals.

Code:

```python
from typing import Iterable, List
def add_item(data: Iterable[int], x: int) -> List[int]:
    """Return a new list with x appended (no mutation)."""
    return list(data) + [x]
if __name__ == "__main__":
    data = []
    data = add_item(data, 10)
    data = add_item(data, 20)
    print(data)
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\C
cripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-202
' 'D:\3-2\AI Assitant Coding\Code Files\Task-13.1.py'
[10, 20]
PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation rules.

• Focus Areas:

o Readability

o Testability

Legacy Code:

```
password = input("Enter password: ")
if len(password) >= 8:
    if any(c.isdigit() for c in password):
        if any(c.isupper() for c in password):
            print("Valid Password")
        else:
            print("Must contain uppercase")
    else:
        print("Must contain digit")
else:
    print("Password too short")
```

Expected Outcome:

o Separate validation functions with clear responsibility

Prompt: simplify and modularize complex validation rules.

• Focus Areas:

o Readability

o Testability

Expected Outcome:

o Separate validation functions with clear responsibility

Code:

```python
def is_long_enough(password: str, min_len: int = 8) -> bool:
    return len(password) >= min_len
def has_digit(password: str) -> bool:
    return any(c.isdigit() for c in password)
def has_upper(password: str) -> bool:
    return any(c.isupper() for c in password)
def validate_password(password: str) -> tuple[bool, list[str]]:
    checks = (
        (is_long_enough, "Password too short (min 8)"),
        (has_digit, "Must contain digit"),
        (has_upper, "Must contain uppercase"),
    )
    errors = [msg for check, msg in checks if not check(password)]
    return (not errors, errors)
if __name__ == "__main__":
    password = input("Enter password: ")
    valid, errors = validate_password(password)
    if valid:
        print("Valid Password")
    else:
        for err in errors:
            print(err)
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\(
cripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-20
' 'D:\3-2\AI Assitant Coding\Code Files\Task-13.1.py'
Enter password: Snehith@123
Valid Password
PS D:\3-2\AI Assitant Coding\Code Files>
```